Deston Willis

Dr. Forouraghi

Research Project

24 April 2026

Final Report

Initial Changes:

In order to test how well the grading AI can grade the students' code, I first modified my app so that it could grade multiple students' code at a time instead of just one. I did this by having the app put all of the student code in a loop and invoke the app for each one. In order for the user to see how far along the model is in evaluating the students' code, in the spinner, I display how many code files have been evaluated out of the total amount of files that were uploaded As well, I put a warning in place so that the code can't be evaluated unless there is a rubric uploaded since doing so would crash the app. This is the code the changes for the Streamlit app:

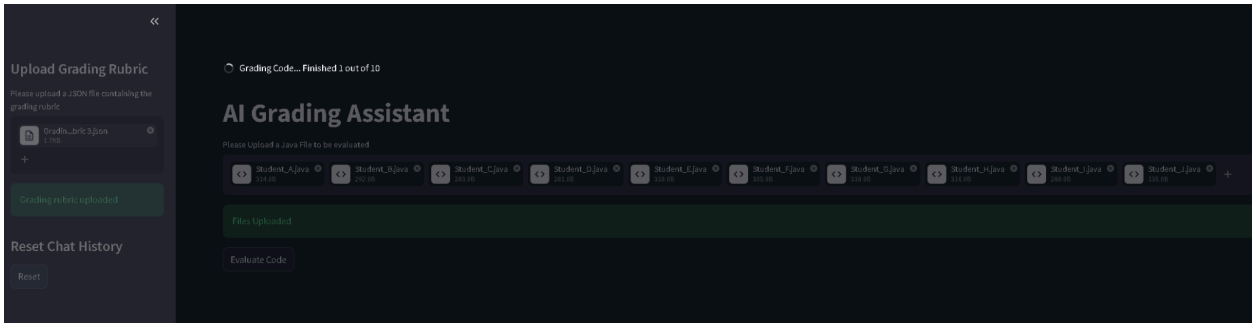
Streamlit App Code:

```python
# Tells the model to run and generate a response
def run_model_button():  1 usage  new *
    files_finished = 0
    files_total = len(file_input)
    for file in file_input:
        with st.spinner("Grading Code... Finished " + str(files_finished) + " out of " + str(files_total)):
            result = app.invoke(
                {
                    "messages": [HumanMessage(content="Evaluate the submitted Java file using the required phases.")],
                    "rubric": rubric_text,
                    "code": file.getvalue().decode("utf-8"),
                    "file_name": file.name,
                    "compiler_runs": 0,
                    "last_compile_result": "",
                    "phase": "compile_original",
                    "working_code": file.getvalue().decode("utf-8"),
                    "test_harness_code": "",
                    "compiler_results": [],
                },
                {"recursion_limit": 10},
            )
            # Displays the LLM's response
            final_answer = result["messages"][-1].content
            st.session_state.chat_history.append(AIMessage(final_answer))
            files_finished = files_finished + 1

# Sidebar for Reset button and rubric upload
with st.sidebar:
    st.title("Upload Grading Rubric")
    # Uploads and stores a JSON rubric
    rubric_file = st.file_uploader(label="Please upload a JSON file containing the grading rubric", type=["json"])
    if rubric_file:
        rubric_data = json.load(rubric_file)
        rubric_text = json.dumps(rubric_data, indent=2)
        st.success("Grading rubric uploaded")
    # Reset button
    st.title("Reset Chat History")
    st.button("Reset", on_click=reset_button)

# Chat Window
# Creates a place for users to upload files and a button to start the model's evaluation
file_input = st.file_uploader(label="Please Upload a Java File to be evaluated", type=["java"], accept_multiple_files=True)
if file_input:
    st.success("Files Uploaded")
    if not rubric_file:
        st.warning("There is no rubric.")
    else:
        st.button("Evaluate Code", on_click=run_model_button)
```

Screenshot of Current Streamlit App:



Rubric and Test File Creation:

Next I created 10 test Java files that contain Java code of varying quality to simulate student code written for an assignment as well as a rubric laying out the requirements for the assignment. The code contains different combinations of syntax and/or logical errors so that there is a wide variety of types of code that the AI needs to evaluate. The assignment is to write a function that takes a double as a parameter and returns the rounded absolute value of the doubled number. The rubric is a JSON document that lays out the different grading criteria and how many points each section is worth. This is the JSON rubric that was used (Grading_Rubric3.json):

Grading Rubric:

```
{
  "criteria": [
    {
      "name": "Compilation",
      "points": 40,
      "description": "The original code compiles and runs without errors. 10 points are deducted from compilation if there are any syntax error in the original code",
      "actual code output": ""
    },
    {
      "name": "Test 1",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": 5,
          "expected_output": 10.0
        },
      "actual code output": ""
    },
    {
      "name": "Test 2",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": -2,
          "expected_output": 4.0
        },
      "actual code output": ""
    },
```

```
    {
      "name": "Test 3",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": 3.5,
          "expected_output": 7.0
        },
      "actual code output": ""
    },
    {
      "name": "Test 4",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": 7.8,
          "expected_output": 16.0
        },
      "actual code output": ""
    },
    {
      "name": "Final Grade",
      "points": 100,
      "description": "This is the total points based on the points from the other criteria. The final grade is out of 100 points. Calculate it by adding Compilation earned points + Test1 Earned Points + Test2 Earned Points + Test3 Earned Points + Test4 Earned Points",
      "final grade": ""
    }
  ]
}
```

This rubric has 5 sections: Compilation and Test 1 through 4. Each section has a name, points (how many points the section is worth), description, and actual code output variable. As well, each test has a test variable that contains the input and expected output for each test. The Compilation section receives full points (40) if the code compiles and runs successfully while having no syntax errors. If there are syntax errors that the AI detects and fixes to make the code work, then 10 points are deducted from this section. Each of the 4 tests are 15 points and receive

full credit if the code passes the test and receive 0 points if the test fails. Each of these tests relate to different aspects of the code. The first test is with a normal positive double, the second test is with a negative double to test that the absolute value of the input is being calculated, and the third and fourth test are with doubles that test that the number is rounded correctly both up and down. As well, there is a final grade section that contains the total points for the assignment.

Testing the Grading App:

To test the app, I uploaded the rubric and the 10 test files and had the LLM evaluate the code and give each a grade and feedback. I used the " qwen2.5-coder:latest" model for my app. The model's responses contained the name of the file, syntax feedback that included how many syntax errors there were and a list of them, test feedback that gives details for each of the tests and whether they passed or not, grade breakdown which lists out how many points were earned for each section of the rubric, and the final grade. After receiving responses for each file's code, I created a table that contains the results of these tests. The table includes the student letter (labeled A-J), some details regarding the code including what syntax and/or logical issues it had, the expected grade based on what I would have given based on the rubric, the actual grade the AI gave, and a short analysis of how the expected grade compares to the actual grade. These are the results:

Results:

Student	Code Specification	Expected Grade	Actual Grade	Short Analysis
A	Correct Code	100	100	The grading AI behaves as expected since the Actual Grade matches the expected grade. The AI gives a grade of 100 since the code functions correctly and doesn't contain any syntax errors.

B	Function is missing 2 semicolons	90	90	The grading AI behaves as expected since the Actual Grade matches the expected grade. The AI gave a grade of 90 since the code functions correctly but contains 2 syntax errors that had to be fixed.
C	Code doesn't round number	70	70	The grading AI behaves as expected since the Actual Grade matches the expected grade. The AI gave a grade of 70 since the code failed 2 tests which resulted in 30 points being lost.
D	Code doesn't take absolute value of number	85	85	The grading AI behaves as expected since the Actual Grade matches the expected grade. The AI gave a grade of 85 since the code failed 1 test and 15 points were deducted.
E	Correct Code with some different names for variables compared to Student A	100	100	The grading AI behaves as expected since the Actual Grade matches the expected grade. The AI gives a grade of 100 since the code functions correctly and doesn't contain any syntax errors.
F	Code returns int instead of double	0	45	The grading AI fixes a non syntax error, which allows the code to run when I didn't expect it too, which is why the expected grade is 0. The AI also inconsistently grades the test. Since the expected output is a double, none of the tests should have passed since the actual output was integers, but one of the tests passed, while two others failed all under the same circumstances of the actual output being the integer version of the expected double output.
G	Code uses ceiling function instead of	85	85	The grading AI behaves as expected since the Actual Grade matches the

	rounding			expected grade. The AI gave a grade of 85 since the code failed one of the tests and lost 15 points.
H	Code has incorrect variable name in return statement and uses floor function instead of rounding	75	75	The grading AI behaves as expected since the Actual Grade matches the expected grade. The AI gave a grade of 75 since the code failed one of the tests and contained a syntax error which resulted in 25 points being lost in total.
I	Function doesn't double number	40	30	The grading AI takes off 10 points for syntax errors, even though there weren't any. While the code does have logical errors which is why all of the tests failed (input value isn't doubled), this shouldn't be penalized as a syntax error since it doesn't stop the code from compiling and running.
J	Code doesn't contain a function or print statement and just calculates value for a hard-coded number.	40	100	I expected all of the tests to fail since there isn't a function to place the test values into but the AI most likely put the values in place of the hard-coded value that is declared and used that as the output, which would technically be the correct number. This may be why all of the tests passed the AI gave a grade of 100.

Analysis:

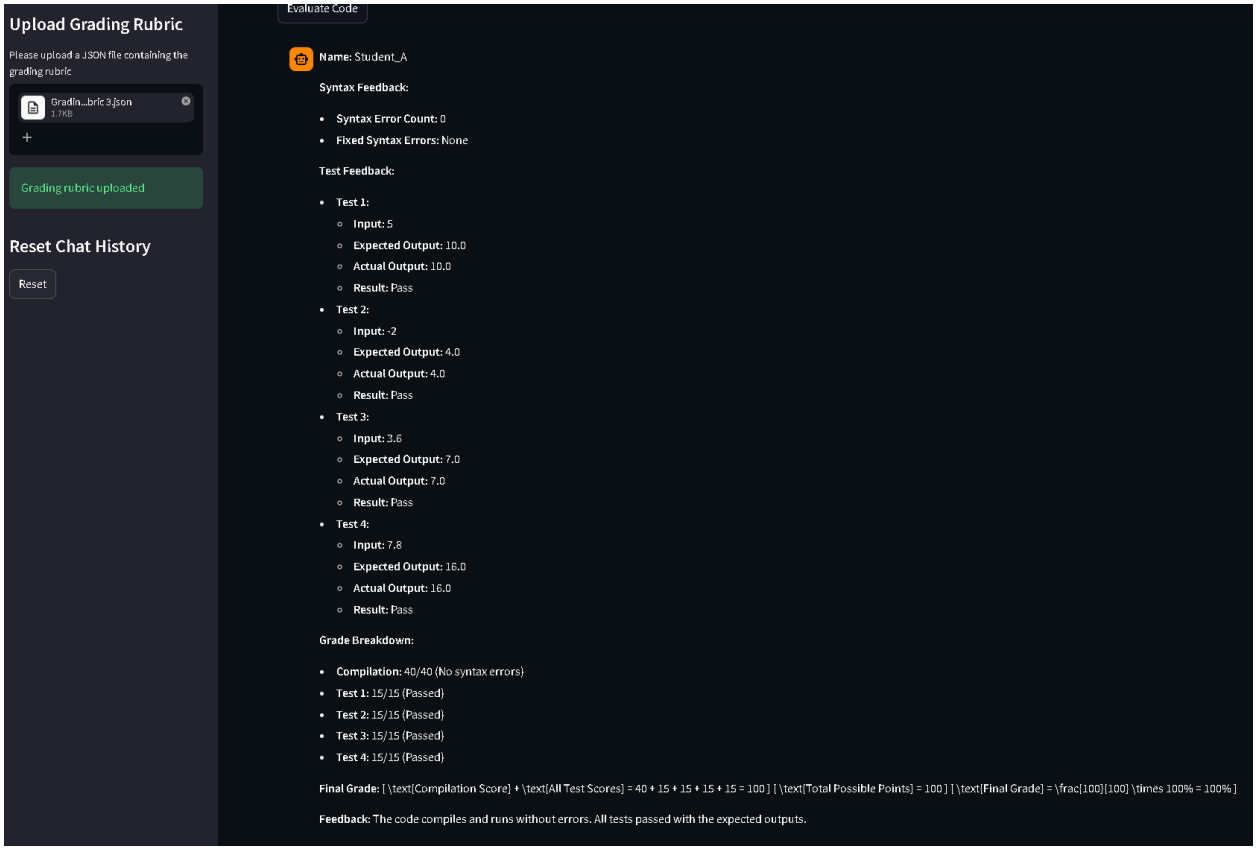
For the most part, the grading AI performed pretty well. The AI was able to respond with a grade for each of the files and each response had all of the sections that I had described before (syntax feedback, test feedback, grade breakdown, and final grade). Out of the 10 files that had to be evaluated and graded, the AI correctly graded 7 of them. Below are two examples of

correctly graded code, one that has scored a 100 (Student A) and one that failed 2 tests and scored a 70 (Student C).

Student A Code:

```
import java.lang.Math;
public class Student_A {
    public static void main(String[] args) {
        double num = 10;
        System.out.println("New Number: " + doubleNumber(num));
    }
    public static double doubleNumber(double n) {
        n = n * 2;
        return Math.round(Math.abs(n));
    }
}
```

Student A Grade:



Student C Code:

```
public class Student_C {
    public static void main(String[] args) {
        double num = 8.1;
        System.out.println("New Number: " + doubleNumber(num));
    }
    public static double doubleNumber(double n) {
        n = n * 2;
        return Math.abs(n);
    }
}
```

Student C Grade:

Upload Grading Rubric

Please upload a JSON file containing the grading rubric

Gradin...bric 3.json
1.7KB

Grading rubric uploaded

Reset Chat History

Reset

Name: Student_C

Syntax Feedback:

- **Syntax Error Count:** 0
- **Fixed Syntax Errors:** None

Test Feedback:

- **Test 1:**
  - **Input:** 5
  - **Expected Output:** 10.0
  - **Actual Output:** 10.0
  - **Result:** Pass
- **Test 2:**
  - **Input:** -2
  - **Expected Output:** 4.0
  - **Actual Output:** 4.0
  - **Result:** Pass
- **Test 3:**
  - **Input:** 3.6
  - **Expected Output:** 7.0
  - **Actual Output:** 7.2
  - **Result:** Fail
- **Test 4:**
  - **Input:** 7.8
  - **Expected Output:** 16.0
  - **Actual Output:** 15.6
  - **Result:** Fail

Grade Breakdown:

- **Compilation:** 40/40 (No syntax errors)
- **Test 1:** 15/15 (Pass)
- **Test 2:** 15/15 (Pass)
- **Test 3:** 0/15 (Fail)
- **Test 4:** 0/15 (Fail)

Final Grade:

- **Calculation:** (40 + 15 + 15 + 0 + 0) / 100 = 70 / 100
- **Total Points Earned:** 70
- **Total Possible Points:** 100
- **Percentage:** 70%

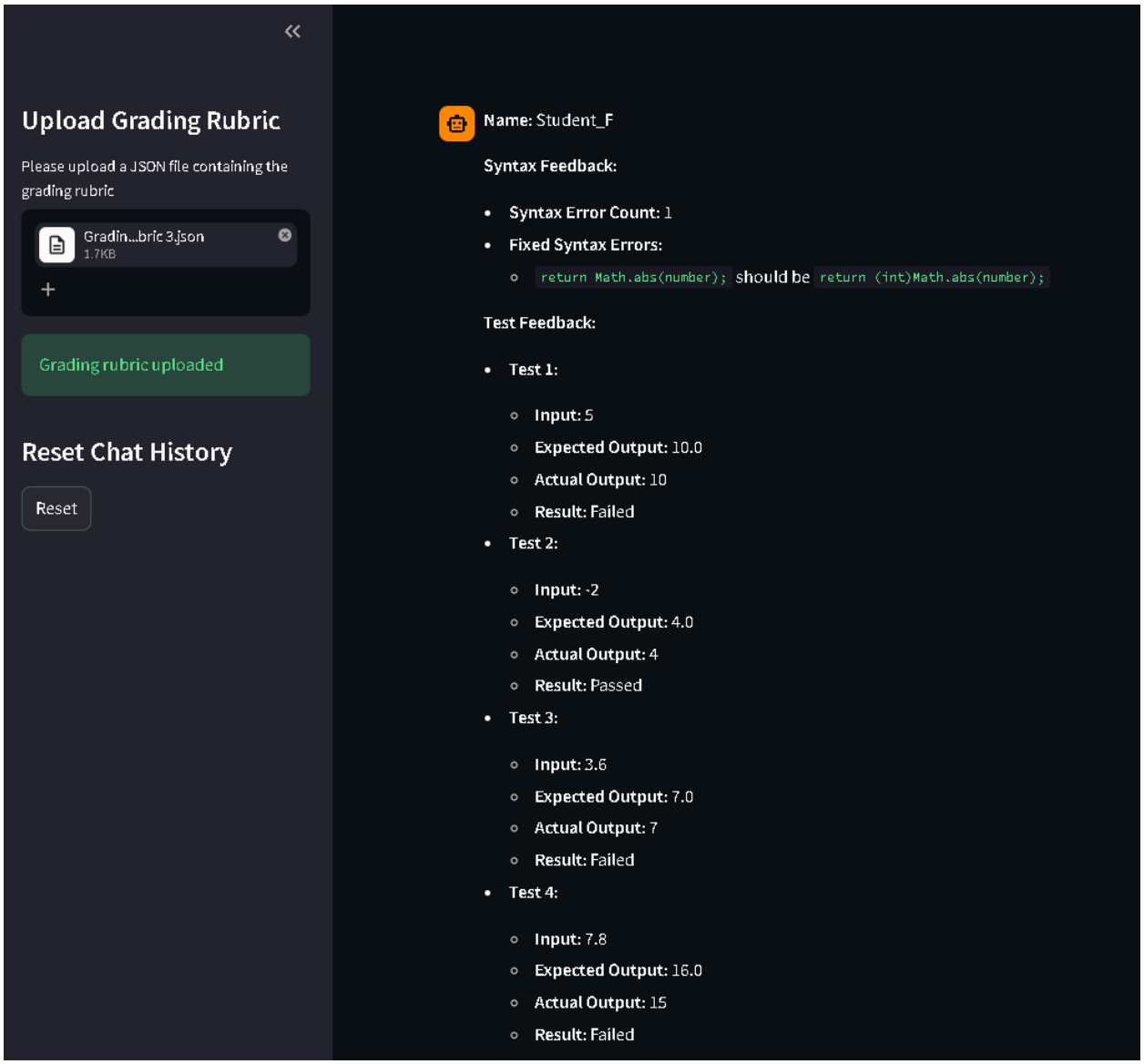
Final Grade: 70/100 (70%)

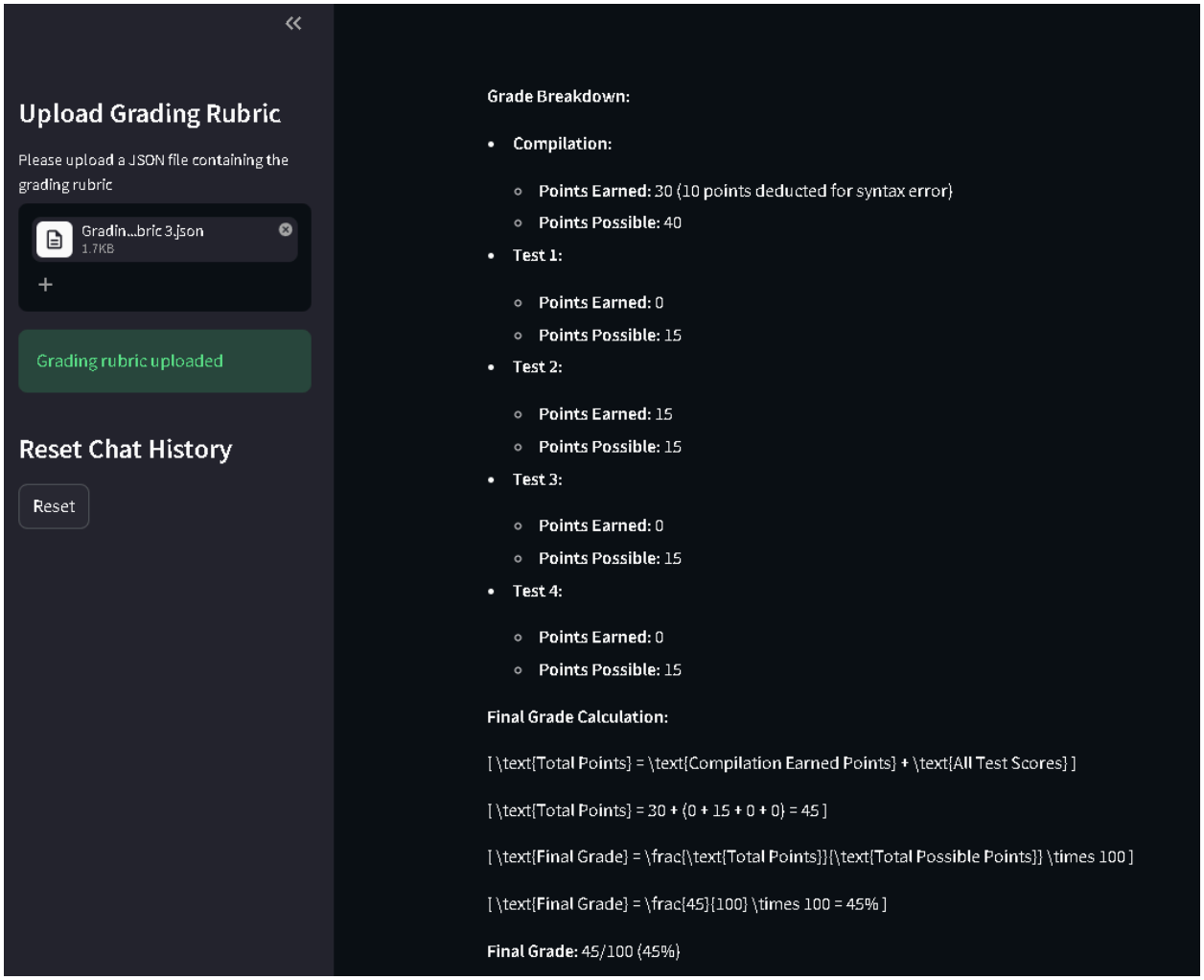
There were only 3 cases where the grading AI gave an incorrect grade, which were Students F, I, and J. Analyzing these cases, the grading AI gave the incorrect grade for different reasons for each of these students. All 3 of these cases are analyzed below.

Student F Code:

```
public class Student_F {
    public static int doubleNumber(double number) {
        number = number * 2;
        return Math.abs(number);
    }
    public static void main(String[] args) {
        double number = 10.4;
        System.out.println("New Number: " + doubleNumber(number));
    }
}
```

Student F Grade:





Student F Analysis:

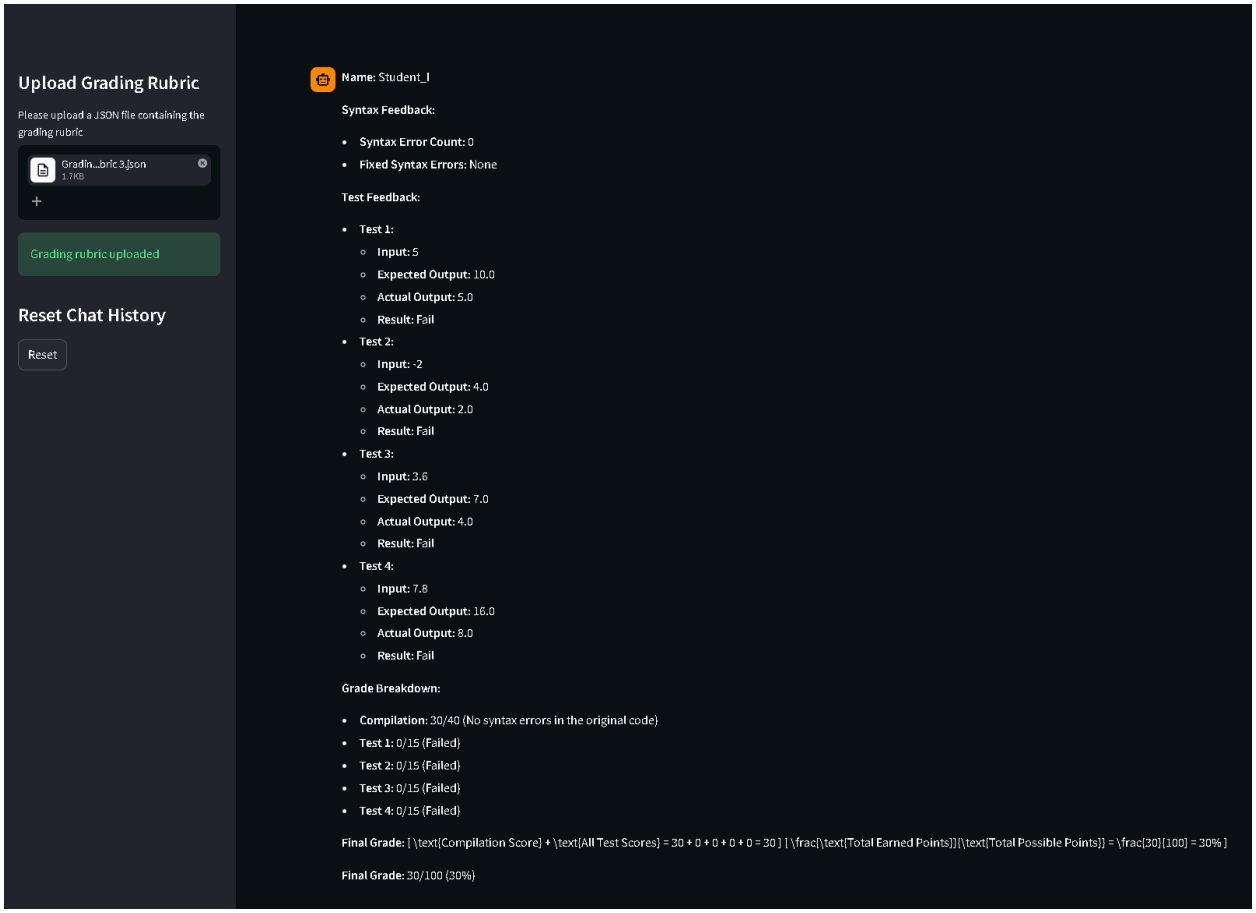
For Student F, there were 2 issues with the grading. The first is that when evaluating the code, the AI fixed an error that wasn't a syntax error. In this case, the AI changed the type of the number to be returned from double to int to fix the code. While this was an issue that prevented the code from running, it shouldn't have been fixed since it wasn't specifically a syntax error and the AI should have only fixed syntax errors. This is why the expected grade is a 0 since the code should have never run since the error wasn't a syntax error and as a result, the compilation and all of the tests would have been unsuccessful. While the system prompt specifically stated to fix only syntax errors, the AI most likely thought that it should fix all errors that prevent compilation

and execution of the code so it fixed the error anyway and then deducted 10 points as if it was a syntax error. The other issue with the grading was that the passing and failing of the tests were inconsistent. Since the values returned were ints, in theory, while one test definitely should have failed since the actual output didn't match the expected output (Test 4), the other 3 should have either all passed since the output values were the same number except the actual output was an int and the expected output was a double or all failed since the types were different. However, Test 2 was passed, while Test 1 and 3 failed despite the circumstances being the same. I am not sure why the tests behaved like this, since I can't find a reason why one of the tests behaved differently from the other two.

Student I Code:

```
public class Student_I {
    public static void main(String[] args) {
        double number = 10;
        System.out.println("New Number: " + doubleNumber(number));
    }
    public static double doubleNumber(double number) {
        return Math.round(Math.abs(number));
    }
}
```

Student I Grade:



Student I Analysis:

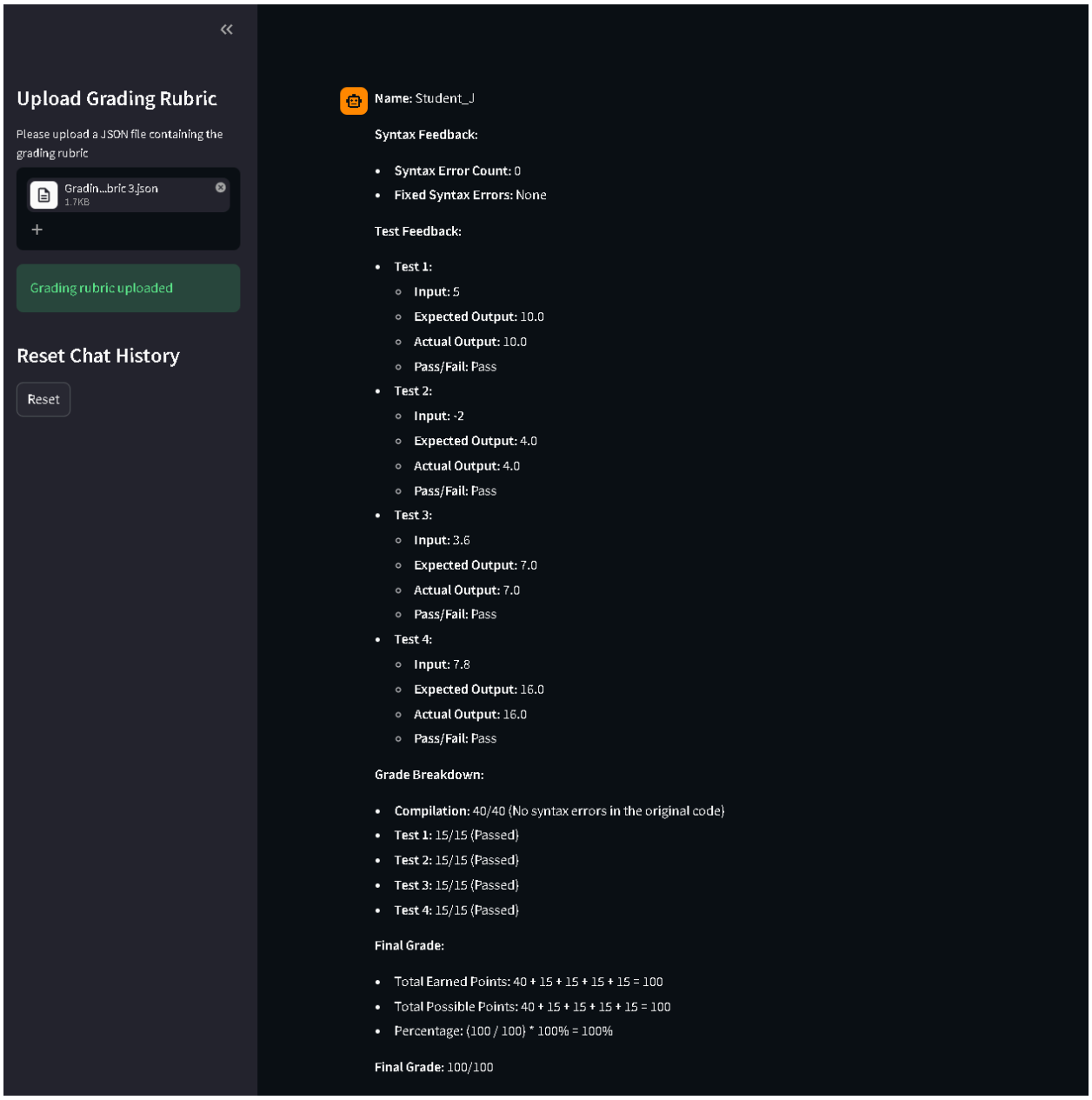
This grade is correct in terms of the tests. All of the tests fail, which is expected since the input value doesn't get doubled in the function. However, for the Compilation section, the AI deducts 10 points for syntax errors even though there aren't any, which resulted in a grade of 30 instead of the expected grade of 40. One possible but unlikely reason that this happened is because the AI got confused and counted the logical error of the numbers not being doubled as a syntax error that should result in points being lost but didn't list it as a syntax error since it isn't one and didn't prevent the code from compiling or executing. A more likely reason that this happened is because the model hallucinated the syntax error when calculating the grade.One

explanation for the hallucination is that the model assumed that there had to have been some sort of syntax error since all of the tests failed, even if there were none counted or listed.

Student J Code:

```
public class Student_J {
    public static void main(String[] args) {
        double number = Math.round(Math.abs(12.4));
    }
}
```

Student J Grade:



Student J Analysis:

This grade was incorrect because the tests that the AI gave all passed when I expected them to all fail, which would have resulted in a 40 instead of the 100 that the AI gave. The reason I expected the tests to fail was because there was no function to input the values into. All the code does is perform the required calculation on a hard-coded number instead of on an input

variable of a function. While I expected the tests to fail since the model would have no place to put the input data for the tests without a function, all of the tests passed, which gave this student a perfect score. One possible reason for this is that when the AI created tests for the code, it simply replaced the hard-coded value with the input values and then treated that as the actual output value. While the assignment's intention is for the students to write a function that does the required calculation, there is nothing anywhere in the rubric or any of the system prompts that states that the code must contain a function. In this case, while the AI didn't produce the expected grade, this seems like less of a problem with the AI and more of a problem of not being specific enough with the instructions and requirements.

Final Analysis/Conclusion:

Overall, the grading AI performed pretty good when grading these 10 student assignments. The AI was able to produce a grade for all of the files, with AI grading 7 out of 10 of them correctly and 1 of the 3 incorrect ones most likely not being wrong because of the AI itself. The AI seemed the best when it was given correct code and could more reliably give accurate grades in these cases. However, while the AI did perform very well when the code was correct, its weak points seemed to be when there were issues with the code and assessing and evaluating those issues correctly. In particular, the model seems to struggle with determining what was and wasn't a syntax error versus other types of errors and also applying penalties, especially for having syntax errors correctly. Another smaller issue with the grading AI that is unrelated to the grades themselves is that the responses vary in structure instead of all looking the same. While all of the responses had the required sections, some responses displayed the information differently than others, which can be seen in some of the previous examples where for example, some final grades display the percentage next to the point total while others don't.

If changes were made to make the AI more reliable when dealing with code with syntax errors and errors in general and with how it formats the responses so that they are uniform, then this app would be able to do a pretty decent job of grading student code.